UniDebugger-Local: Final Report

Stanley Chung (syc2153)

Abstract

Software debugging remains a labor-intensive bottleneck in CI/CD pipelines. While Large Language Models (LLMs) show potential, isolated prompting often fails for complex, repo-level bugs. This project proposes UniDebugger-Local, a rebuild of the UniDebugger framework. By utilizing a 3-level hierarchical coordination of 7 specialized agents, this framework mimics human cognitive processes. To optimize for cost, this version replaces the external search-based Helper agent with local static analysis and documentation retrieval, ensuring the only variable cost is the LLM API.

1. Implementation Changes

1.1 Helper Agent: Tavily Web Search → Tree-sitter Local RAG

Change Summary

The Helper agent previously called the paid Tavily Search API to retrieve web-based debugging references. This has been replaced with a fully local retrieval-augmented generation (RAG) pipeline using Tree-sitter.

Motivation

- Tavily API incurs per-query cost; the project goal is zero external cost beyond the LLM API itself.
- Local symbol retrieval over the actual buggy repo is more targeted than general web search results.

1.2 Locator Agent: GZoltar Spectrum-Based Fault Localization (SBFL)

Change Summary

The Locator agent now receives a ranked list of suspicious line numbers (Ochiai scores) prepended to its prompt, anchoring bug localization before the LLM reasons about the code. The Locator also supports on-demand tool calls: it can request the full coverage report or the body of any named method via function calling, retrieving context only when needed rather than pre-loading everything.

Motivation

SBFL gives the LLM a data-driven prior on which lines are most likely buggy, reducing the search space and improving precision of `// buggy line` marks.

Tool

GZoltar 1.7.3 — https://github.com/GZoltar/gzoltar/releases/tag/v1.7.3

Validation

On BITCOUNT (QuixBugs): stack-trace SBFL ranked line 16 with score 1.0000. The actual bug is on line 16 (`n &= n - 1` should be `n &= n + 1`).

1.3 Slicer Agent: SpotBugs Static Analysis (replaces SonarQube CE)

Change Summary

The Slicer agent now receives a line-range hint from SpotBugs static analysis (or method-range extraction from the stack trace) to narrow down to the specific code slice.

Tool Decision: SpotBugs 4.8.6 (replacing SonarQube CE)

SonarQube CE was replaced with SpotBugs for the following reasons:

Criterion	SonarQube CE	SpotBugs
Deployment	Requires running server (PostgreSQL/H2, web UI)	Single self-contained JAR
Startup latency	30–60 seconds	Immediate
Bytecode analysis	Yes	Yes
Algorithmic bug fallback	None	Method-range from stack trace

Tool

SpotBugs 4.8.6 — https://github.com/spotbugs/spotbugs/releases/tag/4.8.6

Validation

On BITCOUNT (QuixBugs): SpotBugs found 0 bug patterns (expected — algorithmic bug). Method-range fallback returned lines 12–17 (actual bug on line 16). Hint prepended to Slicer: "Suspected region: lines 12-17 [stack-trace method extraction]"

1.4 Summarizer Agent: Stub Extraction

Change Summary

To reduce the token overhead of the Summarizer agent, Java source files are pre-processed using javalang's AST parser before being sent to the LLM. Instead of passing full method bodies, extract_method_stubs() in myast.py traverses all ClassDeclaration and InterfaceDeclaration nodes and emits only the structural skeleton: class names, constructor signatures, and method signatures (return type, name, and typed parameters), with generics and array dimensions preserved.

This strips out all implementation details — loop bodies, conditionals, local variables — that carry no signal for the Summarizer's task of describing what each function does. Since the Summarizer's output format (<ClassName>~<FunctionName>~<params>~<return type>~<description>) is fully derivable from signatures alone, no task-relevant information is lost. If javalang fails to parse the file (e.g., unsupported Java syntax), the function falls back transparently to the original source.

1.5 Cross-File Bug Support Enhancements

Five targeted changes were made to address bugs that span multiple source files.

a) Multi-File Patch Generation

The Fixer and FixerPro prompts now explicitly instruct the LLM to produce one diff hunk per file when a bug spans multiple files, each enclosed in its own code block. The patch parser (split_multi_file_patch in patch.py) splits these into per-file hunks and applies them independently.

b) On-Demand Method Body Retrieval (Tool Use)

Both the Locator and Fixer agents now receive a get_method_body function-calling tool backed by LocalRAG. Instead of pre-loading all related file content (which often exceeds token limits), the LLM can request the full source of any named method on demand, fetching cross-file context precisely when needed.

c) Post-Locator Buggy-Line Enrichment

After the Locator marks buggy lines, the Fixer automatically scans those lines for method calls and retrieves the full bodies of the called methods from LocalRAG. These bodies are appended to the Fixer's context message, ensuring that cross-file call targets are visible at fix time without requiring an explicit tool call.

d) FixerPro Patch Visibility Fix

FixerPro was not receiving the candidate patch text in its prompt due to a missing string append. This meant FixerPro was analyzing a blank patch rather than the actual L1/L2 candidate. The fix ensures the plausibility label and patch text are correctly prepended to the core message.

e) L2/L3 Activation Only on Previous Level Failure

Previously, L2 always ran regardless of L1 success, wasting API budget. L2 and L3 now only activate when the previous level fails to produce a plausible patch.

---

2. Agent I/O Summary

Summarizer

System prompt	"Summarize every class and function"
User input	Stub-extracted Java source (method signatures only, no bodies)
Expected output	One line per function: <ClassName>~<FunctionName>~<params>~<return type>~<description>
Parsed into	Nested dict {class → {func → {paras, return_type, desp}}}

RepoFocus

System prompt	"Identify relevant files"
User input	Imported packages, failing test cases, directory tree, optional coverage report
Expected output	List of valid file paths + === block explaining relevance
Parsed into	List of 2–6 valid file paths (test paths filtered out)

Helper

System prompt	"Write a debugging guide grounded in local repo symbols"
User input	Buggy code, failing tests, top-5 LocalRAG symbols
Expected output	Step-by-step debugging guide + `===` block with `path/to/File.java:linenum`
Parsed into	`{aim: guide text, exp: file references}`

Slicer

System prompt	"Extract the suspicious 50–100 line code snippet"
User input	SpotBugs hints, Summarizer output, Helper guide, failing tests, coverage
Expected output	Java code snippet + `===` block with investigation suggestions
Parsed by	Mapping snippet back to original file offsets (±10 lines of context)

Locator

System prompt	"Mark buggy lines with comments"
User input	SBFL rankings, Summarizer/Helper output, failing tests, Slicer snippet
Tools available	`get_method_body` (fetches full source of any method), `failing_coverage` (injects coverage)
Expected output	Code with `// buggy line` or `// missing code:[...]` annotations + `===` fix advice

Parsed by	Aligning annotations to original source lines

Fixer / FixerPro

System prompt	Labeled/unlabeled variant (Fixer); plausible/not-plausible variant (FixerPro)
User input	Coverage, Summarizer, Helper, failing tests, Locator-annotated code; called method bodies appended automatically
Tools available	get_method_body (Fixer only)
Expected output	Git diff patch in backticks + === block explanation; one hunk per file for multi-file bugs

Output dictionary shape (all agents):

```
None
{
  "aim": <primary extracted content>,  # passed to the next agent
  "exp": <explanation text>,
  "ori": <full raw LLM response>
}
```

---

3. About the Defects4J Benchmark

Defects4J (Just et al., 2014) is a curated database of real-world bugs extracted from the version control history of open-source Java projects. The bugs were not synthesized or injected artificially — they are actual defects that were introduced by developers during normal development, discovered through testing or user reports, and subsequently fixed in a later commit.

How bugs were collected: The Defects4J researchers mined the Git and SVN histories of widely-used Apache and other open-source projects. For each candidate commit flagged as a bug fix (identified by keywords such as "fix", "bug", "issue" in the commit message and cross-referenced with project issue trackers such as JIRA and Bugzilla), they manually verified that:

1. The commit is a genuine bug fix, not a feature addition or refactoring.

2. At least one test case exists that fails on the buggy version and passes on the fixed version.
3. The bug is reproducible — the project can be checked out at the exact buggy revision and the failing test confirmed.

Each entry in Defects4J packages the buggy source, the fixed source, and the triggering test suite together, making it possible to evaluate a repair tool in a controlled and reproducible way.

Why this matters for evaluation: Because the bugs are real and historical, the benchmark reflects the true distribution of defects that developers encounter — including subtle logic errors, off-by-one mistakes, incorrect null handling, and interface contract violations across multiple files. The developer's original patch serves as the reference fix, and a generated patch is considered correct if it passes the full test suite (not just the triggering tests), reducing the risk of overfitting to specific test inputs.

The projects in this evaluation (Chart, Time, Math, Cli, Lang, Mockito, JxPath) accumulated their bugs over years of active development, ranging from short-lived regressions to long-standing defects that survived multiple releases before being caught.

---

4. Defects4J Evaluation Setup

Bug Selection

100 bugs were selected across 7 projects to cover a mix of single-file and cross-file bugs:

Project	Bugs Selected	Count	Cross-file
Chart	1–26	26	2 (14, 18)
Time	1–27 (excl. deprecated 21)	26	4 (1, 2, 12, 26)
Math	1, 4, 6, 14, 22, 71, 77, 98	8	8 (all)
Math	2, 3, 5, 7, 8, 9, 10, 11	8	0
Cli	13, 16, 21, 30, 31, 34, 36	7	7 (all)
Lang	1, 3, 4, 5, 6, 7, 9, 10, 13, 16, 21, 24, 27, 29, 33	15	0
Mockito	14, 16, 17, 19, 30	5	5 (all)
JxPath	1, 4, 7, 9, 11	5	5 (all)

Total	**100**	**31**

Cross-file classification is based on the number of files modified in the developer's reference patch (> 1 file = cross-file).

Setup Procedure

Each bug was checked out using the Defects4J CLI (defects4j checkout), compiled, and tested to confirm at least one failing test exists. The buggy source file path for each bug is recorded in benchmarks/d4j/root_cause_path.json, derived from modified_classes/{id}.src in the Defects4J framework directory.

Infrastructure notes:

- Java 11 required; detected automatically via /usr/libexec/java_home -v 11
- Defects4J's Perl CLI requires String::Interpolate, installed via cpanm; PERL5LIB injected into all subprocesses
- Chart bugs use SVN (not Git); Subversion installed via Homebrew
- Time bug 21 is deprecated (JVM8.Not.Reproducible) and excluded

Plausibility is determined by: applying the patch → defects4j compile → defects4j test → checking Failing tests: 0.

---

5. Evaluation Results

5.1 QuixBugs Results

Benchmark	QuixBugs (40 intentionally buggy Java algorithm programs)
Model	gpt-4o
Metric	Plausible fix rate — patch passes all Gradle tests

Level	Agents	Plausible	Fix Rate	Total Cost	Cost/Bug
L1	Locator + Fixer	30 / 40	75.0%	$0.87	$0.022

L2	L1 + Slicer + Summarizer + FixerPro	34 / 40	85.0%	$2.29	$0.057
L3	L2 + Helper + RepoFocus	36 / 40	90.0%	$2.75	$0.069

Ablation:

- **L1 → L2** (+4 net): 5 newly fixed (MINIMUM_SPANNING_TREE, QUICKSORT, SHUNTING_YARD, SIEVE, SUBSEQUENCES), 1 regression (LCS_LENGTH)
- **L2 → L3** (+2 net): 4 newly fixed (DEPTH_FIRST_SEARCH, LCS_LENGTH, POWERSET, WRAP), 2 regressions (QUICKSORT, SIEVE)

Regressions across levels are attributed to LLM non-determinism, indicating the patches are near GPT-4o's decision boundary.

5.2 Defects4J Results

Benchmark	100 curated Defects4J bugs (31 cross-file, 69 single-file)
Model	gpt-4o-2024-08-06
Run mode	All three levels in one pass (L2 only if L1 fails, L3 only if L2 fails)
Total cost	~$22

Results by level:

Level	Bugs Added	Cumulative
L1	17	17 / 100
L2	4	21 / 100
L3	0	21 / 100
Total plausible		**21 / 100 (21%)**

Bugs fixed at each level:

- **L1:** Chart_1, Chart_5, Chart_7, Chart_10, Chart_19, Chart_20, Chart_24, Chart_26, Cli_34, Lang_7, Lang_16, Lang_21, Lang_24, Lang_27, Math_3, Math_8, Time_15
- **L2:** Chart_11, Chart_12, Math_5, Time_11
- **L3:** *(none)*

Results by bug type:

Category	Fixed	Total	Fix Rate
Single-file	20	69	29.0%
Cross-file	1	31	3.2%
Total	**21**	**100**	**21.0%**

The one cross-file fix (Cli_34) came from L1. The large gap between single-file (29%) and cross-file (3.2%) fix rates reflects the difficulty of bugs that require reasoning across multiple source files simultaneously.

---

6. GPT Baseline Comparison

Setup

To quantify how much the multi-agent pipeline adds beyond a vanilla LLM call, a GPT baseline was run using the same model (gpt-4o-2024-08-06) on the same 100 bugs. The baseline sends a **single prompt per bug** containing the buggy code and failing test cases, and asks for a git diff patch directly — no Locator, no Slicer, no RepoFocus, no Helper, no multi-level escalation.

The baseline uses the same fixer.yaml unlabeled system prompt as Unidebugger's Fixer agent, and the same patching_and_testing_d4j() function to evaluate plausibility. The only difference is the absence of the agent pipeline.

Results

System	Plausible	Single-file	Cross-file	Cost
GPT baseline (1 call/bug)	5 / 100 (5%)	5 / 69	0 / 31	~$2–3
Unidebugger (3 levels)	21 / 100 (21%)	20 / 69	1 / 31	~$22

Overlap analysis:

- Bugs fixed by **both**: Chart_10, Chart_12, Chart_20 (3 bugs)
- Fixed by **Unidebugger only**: 18 bugs
- Fixed by **baseline only**: Lang_33, Math_7 (2 bugs, attributed to LLM non-determinism)

The pipeline provides a **4.2× improvement** over the single-call baseline (21% vs. 5%).

Analysis

The baseline result confirms that the multi-agent architecture adds substantial value beyond simply prompting GPT with the raw bug information. The Locator's `// buggy line` annotations are the primary driver: L1 (Locator + Fixer) accounts for 17 of the 21 fixes, and the precise localization gives the Fixer a targeted line to fix rather than requiring it to search the entire file.

L3 produced no new fixes on D4J. The cross-file context mechanisms (RepoFocus, Helper, post-Locator enrichment) are most effective when bug symptoms clearly point to an external dependency — a pattern less prevalent in the selected D4J bugs.

---

7. Comparison with Original UniDebugger

System	QuixBugs (Java)	Defects4J (subset)
UniDebugger (original, gpt-4o)	40 / 40 (100%)	not reproduced
UniDebugger-Local (gpt-4o-2024-08-06)	36 / 40 (90%)	21 / 100 (21%)
GPT baseline (gpt-4o-2024-08-06)	—	5 / 100 (5%)

QuixBugs gap (10%): Attributable to the Helper agent replacement (local RAG vs. Tavily web search) and reduced Slicer anchor precision (SpotBugs detects zero patterns on algorithmic bugs).

Defects4J (21%): Consistent with published LLM-based APR results (ChatRepair: ~25–30% on curated D4J subsets). The 4.2× improvement over the GPT baseline demonstrates the value of the multi-agent design.

What This Implementation Demonstrates

- **Hierarchical architecture validated** on both QuixBugs and Defects4J: each level fixes bugs the previous level cannot, matching the ablation trend in the original paper.

- **4.2× improvement over single-call GPT baseline** (21% vs. 5%), confirming that the specialized agent roles (Locator, Slicer, RepoFocus, Helper) provide concrete repair value beyond raw LLM capability.
- **Cross-file bugs remain an open challenge**: 3.2% fix rate vs. 29% for single-file bugs, motivating further work on cross-file context retrieval.
- **Local-tool variant eliminates all external service dependencies** (Tavily, SonarQube) while retaining 90% of the original fix rate on QuixBugs.

---

8. Limitations and Future Work

8.1 Limitations

Cross-file bug fix rate. The system fixes only 3.2% of cross-file bugs (1/31) versus 29% of single-file bugs (20/69). The cross-file context mechanisms — RepoFocus, post-Locator method-body enrichment, and the get_method_body tool — are only effective when the bug's call chain is shallow and clearly surfaced by the stack trace. For bugs that span independent subsystems or require understanding a deep inheritance hierarchy across many files, the pipeline retrieves too little context and the Fixer produces patches scoped to the wrong file.

Three representative failures illustrate the failure modes: (1) **Mockito_14** — the Fixer called ongoingStubbing.setFinishedStubbingNeeded(true), a method that does not exist on OngoingStubbingImpl; the actual fix required coordinated changes to MockHandler and two other classes, none of which were in the Locator's view. This was also the single most expensive run: 1.1M prompt tokens, $2.98, zero plausibility. (2) **Cli_13** — the Locator correctly identified a missing method signature in WriteableCommandLine (interface), but the Fixer added a different, incompatible signature (List getArgumentValues) rather than the correct looksLikeOption(String), because the implementation class that uses it was never fetched. (3) **JxPath_1** — the Fixer applied UNKNOWN_NAMESPACE.equals(namespace) instead of namespace == UNKNOWN_NAMESPACE, a plausible null-safety heuristic, but the real bug was in a different file entirely; the slicer never retrieved that file because the stack trace landed in DOMNodePointer. The sole cross-file success, **Cli_34** ($0.06), succeeded precisely because its fix was a one-line addition to a single interface that the Locator had directly in scope.

L3 provides no new fixes on Defects4J. Despite adding the Helper (local RAG guide) and RepoFocus agents, L3 fixed zero additional Defects4J bugs beyond L2. The Helper agent's debugging guide is most useful when the failing symptom clearly maps to a library function whose semantics can be retrieved from the local repo; on D4J bugs — which include subtle logic errors, incorrect null handling, and contract violations — the guide rarely narrows the search space further than what L1/L2 already established. The added cost of L3 ($0.46 more per run) is therefore not justified on this benchmark.

For example, all five Mockito bugs and all five JxPath bugs failed at every level, including L3. Chart_14 consumed 311K prompt tokens ($0.95) across all three levels and remained implausible; Chart_13, _15,

_16, _17 each exceeded $0.40–$0.95 and similarly failed. The Helper's guide for these bugs either repeated what the Locator had already identified or pointed to code that was already in the Fixer's context — adding tokens but no new signal.

SBFL unreliable on algorithmic and pure-logic bugs. GZoltar's spectrum-based fault localization requires at least one passing and one failing test to compute a coverage differential. On QuixBugs, where bugs are intentionally seeded single-line mutations, this works well (BITCOUNT: score 1.0 on the correct line). On Defects4J bugs where the triggering tests are few or where the fault manifests only under specific input combinations, the Ochiai rankings are noisy and provide weak signal to the Locator.

SpotBugs misses algorithmic and logic errors. SpotBugs performs bytecode-pattern matching (null dereference, resource leak, incorrect API usage) but produces zero findings for semantic or algorithmic bugs — the dominant bug class in both QuixBugs and a significant fraction of Defects4J. The method-range fallback from the stack trace partially compensates, but a missed SpotBugs hit means the Slicer receives a wider and less precise code window.

LLM non-determinism causes inter-level regressions. Four bugs regress across levels (L1→L2: LCS_LENGTH; L2→L3: QUICKSORT, SIEVE, and one D4J bug fixed at L1 but not re-evaluated). These regressions occur on bugs near the model's decision boundary: the patch is marginally correct at a lower level, and the additional context at the next level shifts the generation toward a different (wrong) fix. This non-determinism makes the aggregate fix rate an expectation over random seeds rather than a deterministic bound.

The **LCS_LENGTH** regression is the clearest example. At L1, the Fixer produced a minimal, correct one-line fix: `dp.get(i-1).get(j-1)` instead of `dp.get(i-1).get(j)`. At L2, FixerPro — now seeing the Slicer's snippet, the Summarizer's description, and a "not plausible" label — rewrote the entire function: it changed loop bounds to `i <= s.length()`, switched to `s.charAt(i-1)` indexing, rewrote the merge step as `Math.max(dp.get(i-1).get(j), dp.get(i).get(j-1))`, and returned `dp.get(s.length()).get(t.length())`. This is a standard LCS implementation but uses 1-indexed table semantics that are incompatible with the rest of the program's 0-indexed HashMap structure, causing the test suite to fail. The L1 patch was correct by being minimal; L2's additional context gave the model enough latitude to produce a plausible-looking but wrong rewrite.

Plausible ≠ correct. The evaluation criterion is patch plausibility — passing the full Defects4J test suite. A patch can be plausible but semantically wrong: it might satisfy the test suite by special-casing the failing input while leaving the underlying defect intact. No comparison against the developer's reference patch (exact-match or semantic equivalence) was performed, so the 21% figure is an upper bound on true correctness.

Benchmark scope and language coverage. The evaluation covers 100 bugs from 7 Java projects. Results may not generalize to projects with different coding styles, heavier framework use (e.g., Spring, Hibernate), or to other languages. The tool chain (GZoltar, SpotBugs, javalang, Tree-sitter Java grammar) is Java-specific, and porting to Python or C requires rebuilding each component.

Infrastructure fragility. Correct execution depends on Java 11 (not 8, not 17), a Perl installation with String::Interpolate, Subversion for Chart's SVN history, and specific GZoltar/SpotBugs JAR versions. Any environment mismatch silently degrades the pipeline (e.g., a wrong Java version causes defects4j compile to fail, which the harness records as a non-fix rather than an environment error).

8.2 Future Work

Improving cross-file bug repair. The largest headroom is in cross-file bugs (3.2% → target ≥15%). Promising directions include: (1) building a static call graph over the whole project at checkout time and giving the Locator the full transitive callee chain for each suspicious line, rather than a flat list of retrieved method bodies; (2) integrating class-hierarchy and interface-implementation edges so the Fixer can identify the correct concrete class when a bug resides in an override; (3) using a dependency-aware chunking strategy in the RAG index (chunk at class boundary, embed class-level docstrings) to improve retrieval precision for cross-package calls.

Hybrid fault localization. Combining SBFL (GZoltar Ochiai) with mutation-based fault localization (MBFL) or value-based techniques (e.g., recording variable values at failing-test checkpoints) could improve Locator precision on bugs where coverage differentials are small. For bugs where SpotBugs fires, its bug category (e.g., NP_NULL_ON_SOME_PATH) could be used to select a specialized Locator prompt rather than a generic one.

Deterministic or ensemble patching. To reduce inter-level regressions, future versions could generate k candidate patches per level using temperature sampling, run the full test suite on each, and select the first plausible patch rather than accepting the first-generated one. An ensemble voter (majority-rule over patch hunks) could further stabilize results on borderline bugs without increasing cost linearly.

Correctness evaluation beyond test-suite plausibility. Future evaluation should include patch-correctness assessment: comparing generated patches to the developer reference patch using AST-edit-distance or semantic equivalence checking (e.g., running the patch against a held-out set of tests not used in evaluation). This would separate "lucky" over-fitted fixes from genuinely correct repairs and provide a tighter upper bound estimate.

Scaling to full benchmarks and other languages. Evaluating on the full Defects4J corpus (800+ bugs), SWE-bench (Python, real GitHub issues), and BugsInPy would test generalization. Extending the pipeline to Python would require replacing GZoltar with a Python coverage-based SBFL tool (e.g., fault-localization via pytest-cov), SpotBugs with a Python static analyzer (e.g., Pylint, Pyright), and the javalang stub extractor with an AST-based stub extractor for Python.

Adaptive agent routing. Currently, all bugs go through the same fixed agent sequence. A lightweight classifier trained on bug metadata (stack-trace depth, number of modified files in past fixes for the same project, SpotBugs category) could route bugs directly to the most effective level, skipping L1 for bugs historically requiring L2/L3 and avoiding unnecessary token spend on simple single-line fixes.

Reducing cost while maintaining coverage. The 4.2× improvement in fix rate costs roughly 8–10× more than the baseline. A cost-aware escalation strategy could checkpoint after each level and only escalate when a confidence signal (e.g., Locator uncertainty, patch edit distance from the original) exceeds a threshold. Prompt caching (reusing the Summarizer output across levels) and smaller task-specific models for lower-level agents (e.g., Haiku for the Summarizer and RepoFocus) could reduce per-bug cost without sacrificing L1 fix rate.

The cost disparity in the current results is stark. Chart_10, Chart_20, and Chart_24 — all correctly fixed at L1 — each cost under $0.05. Mockito_14, which failed at every level, cost $2.98; Chart_14 cost $0.95; Cli_30 cost $0.82. The seven cross-file Cli bugs that all failed cost a combined ~$3.04. A classifier that predicted "cross-file bugs with deeply nested interface hierarchies are unlikely to be fixed" and skipped them would have saved ~14% of the total budget with zero impact on the fix count.

Conclusion

UniDebugger-Local extends the original pipeline with Defects4J evaluation and cross-file bug support. On a curated 100-bug D4J selection, the system achieves 21% plausibility — a **4.2× improvement** over a single-call GPT baseline (5%), validating that the multi-agent architecture adds measurable repair capability. The hierarchical escalation property (L1→L2→L3) is confirmed: L2 contributes 4 additional fixes beyond L1, matching the ablation direction reported in the original paper.

Cross-file bugs remain significantly harder (3.2% vs. 29% single-file), pointing to the main open challenge for future work. On QuixBugs, the local-tool variant retains 90% of the original pipeline's fix rate while eliminating all external service dependencies, confirming that the architectural design matters more than the specific tools used.